Here we have two binary values with five contiguous bits set: one with the most significant digits and one with least, which means only bits four and five are set in both. Bitwise AND (`$mostSignificant & $leastSignificant`) positionally compares the bits in each operand and sets the return bits to *1* where both are the same, so we get *00011000*. Bitwise OR (`$mostSignificant | $leastSignificant`) will set the return bits to *1* in each position where either operand has a bit set, so we get *11111111*. Finally, bitwise XOR (`$mostSignificant ^ $leastSignificant`) sets return bits to *1* in the positions where either operand has *1* set, but not both, so we get *11100111*.

Table 7-1. *Bitwise operator effects*

Operator:	**AND &**	**OR |**	**XOR ^**
Input 1:	11111000	11111000	11111000
Input 2:	00011111	00011111	00011111
Result:	00011000	11111111	11100111

Caution needs to be taken when using bitwise operations in expressions that also contain comparisons:

```
$leastSignificant = bindec('00011111');

var_dump($leastSignificant); // int(31)
var_dump($leastSignificant == true); // returns bool(true) which casts
                                     to int(1)
var_dump($leastSignificant ^ $leastSignificant == true); // equivalent to
                                                            00011111
                                                            ^ 00000001
var_dump(($leastSignificant ^ $leastSignificant) == true); // equivalent to
                                                              00000000 == true

Output:
int(31)
bool(true)
int(30)
bool(false)
```